

Universidad de Valladolid

Grado en Informática

Evaluación de Sistemas Informáticos

Práctica 2: Pruebas de carga, Locust y Prometheus

Iván Moro Cienfuegos

ivan.moro22@estudiantes.uva.es

Pablo March Ortega

pablo.march23@estudiantes.uva.es

Nicolás Reyes Gutiérrez

nicolas.reyes@estudiantes.uva.es

Mayo 2026



Universidad de Valladolid

Índice

1. Introducción y Objetivos	3
2. Descarga y Despliegue de las herramientas	3
2.1. Backend web	3
2.2. Prometheus	3
3. Instrumentación con Prometheus	3
3.1. Métricas implementadas	4
3.1.1. Métricas HTTP Generales	4
3.1.2. Métricas Específicas de Negocio (OAuth2)	4
3.2. Justificación del diseño	4
4. Visualización en Grafana	5
4.1. Tráfico del servidor (RPS)	5
4.2. Tráfico por Endpoint	6
4.3. Consumo de CPU del Backend	6
4.4. Latencia P95	7
4.5. % de Errores del Servidor	7
4.6. Emisión de Tokens en tiempo real	8
4.7. Flujos de Autenticación Iniciados	8
4.8. Tasa de Éxito de Autenticación	9
5. Locust para Realización de pruebas	9
5.1. Descarga e información relevante	9
5.2. Pruebas tipo identificadas	10
5.3. Descripción flujos de comportamiento	10
5.4. Configuración general	11
6. Resultados y Evaluación de Rendimiento	12
6.1. Prueba de Carga (50 Usuarios Concurrentes)	12
6.2. Prueba de Estrés y Punto de Ruptura (800 Usuarios)	13
7. Conclusiones Finales	13

1. Introducción y Objetivos

Sabemos que el correcto funcionamiento de un sistema en entornos de desarrollo local no garantiza su supervivencia en producción bajo condiciones de alta concurrencia, por esto, las pruebas de carga y estrés son fundamentales. Este tipo de pruebas nos sirve para identificar cuellos de botella, conocer los rangos operacionales del hardware subyacente y definir estrategias de escalabilidad.

En esta segunda práctica, se aborda la monitorización y evaluación exhaustiva de un backend desarrollado en Python (Flask) que implementa un servidor de autenticación OAuth2. Mediante la instrumentación del código, la exposición de métricas para **Prometheus**, la visualización analítica en **Grafana** y la inyección progresiva de carga con **Locust**, se busca (entre otras cosas) establecer el límite operacional del sistema (RPS) y garantizar que los tiempos de respuesta, concretamente el Percentil 95, se mantengan dentro de umbrales aceptables.

2. Descarga y Despliegue de las herramientas

Los primeros pasos de la práctica fueron preparar todo el entorno, creando el directorio `/home/usuario/Practica2` para meter en él todo lo necesario para este segundo ejercicio.

2.1. Backend web

Para tener el backend proporcionado (OAuth2) en la máquina virtual y poder instrumentarlo, lo descargamos y lo pasamos a la carpeta `Practica2` de la máquina virtual mediante el comando `scp` de la terminal, para después descomprimirlo.

2.2. Prometheus

Empezamos por descargar Prometheus, para luego extraerlo en la máquina virtual y generar el fichero de configuración `"prometheus.yml"` tal y como se indica en el enunciado del ejercicio (manteniendo el intervalo de pull de datos cada 15 segundos y cambiando `"localhost:XXXX"` por `"127.0.0.1:5001"`). Para finalizar con la configuración de Prometheus, creamos un dashboard en Grafana llamado `Practica2` y agregamos como Data Source Prometheus (con el puerto 9090 tal como se recomienda).

3. Instrumentación con Prometheus

En esta práctica se ha instrumentado el backend OAuth2 proporcionado utilizando la librería `prometheus_client` para Python. Todas las métricas se han centralizado en el módulo `metrics.py`, para evitar así ensuciar el código del negocio.

3.1. Métricas implementadas

3.1.1. Métricas HTTP Generales

- **http_requests_total** (Counter)
Contador que registra el número total de peticiones HTTP procesadas, etiquetado por **method**, **endpoint** y **status_code**. Permite analizar la tasa de peticiones, distribución por rutas y tasa de errores (códigos 4xx/5xx).
- **http_request_duration_seconds** (Histogram)
Histograma que mide la latencia de las peticiones en segundos, etiquetado únicamente por **endpoint**. Este histograma es fundamental porque permite calcular percentiles (p50, p95, p99) de latencia, uno de los indicadores clave de rendimiento en entornos productivos (en este caso el p95 es el que nos interesa).
Esta métrica destaca dado que en el enunciado se nos sugería que hiciéramos uso principalmente de métricas tipo Counter y tipo Gauge, pero dada la cantidad de información relevante que podía aportar, tomamos la decisión de incluirla de todas formas.

```
from prometheus_client import Counter, Histogram

http_request_duration_seconds = Histogram('http_request_duration_seconds', '
    Latencia de las peticiones HTTP en segundos', ['endpoint'])
```

Métrica tipo Histogram `http_request_duration_seconds` (metrics.py)

3.1.2. Métricas Específicas de Negocio (OAuth2)

- **oauth2_flows_total** (Counter)
Registra los intentos de flujos de autenticación, diferenciando por **flow_type** (authorization_code, client_credentials, refresh_token) y **status** (success, failed). Permite monitorizar la salud del sistema de autenticación y detectar posibles ataques de fuerza bruta o problemas de credenciales.
- **oauth2_tokens_issued_total** (Counter)
Contador de tokens emitidos correctamente, etiquetado por **grant_type**. Facilita el seguimiento del volumen de tokens generados por cada tipo de concesión y ayuda a detectar anomalías en el patrón de emisión.

3.2. Justificación del diseño

Las métricas se han diseñado siguiendo las mejores prácticas de observabilidad:

- **RED Method**: Rate (`http_requests_total`), Errors (`status_code 4xx/5xx`) y Duration (`http_request_duration_seconds`).
- **Uso de middleware**: La instrumentación HTTP se realiza de forma automática mediante decoradores `@app.before_request` y `@app.after_request`, evitando contaminar las rutas con código de monitorización.

- **Etiquetado adecuado:** Las etiquetas utilizadas son una potente forma de filtrado a la hora de visualizar gráficas mediante llamadas PromQL en Grafana.
- **Separación de concerns:** Las métricas generales HTTP están separadas de las métricas de negocio OAuth2, permitiendo un análisis tanto a nivel de infraestructura como a nivel de dominio.

La exposición de métricas se realiza en el endpoint `/metrics` mediante `DispatcherMiddleware`, garantizando que Prometheus pueda scrapearlas sin interferir con el funcionamiento normal de la aplicación Flask.

4. Visualización en Grafana

A través del Datasource de Prometheus vinculado a Grafana diseñamos un *Dashboard*, que combinando métricas de sistema y métricas específicas del dominio OAuth2, proporciona una visión integral del estado del backend, utilizando consultas PromQL dinámicas.

Las gráficas incluidas son las siguientes (todas recogidas de la misma ejecución de la prueba de carga con 50 usuarios, de la que hablaremos posteriormente):

4.1. Tráfico del servidor (RPS)

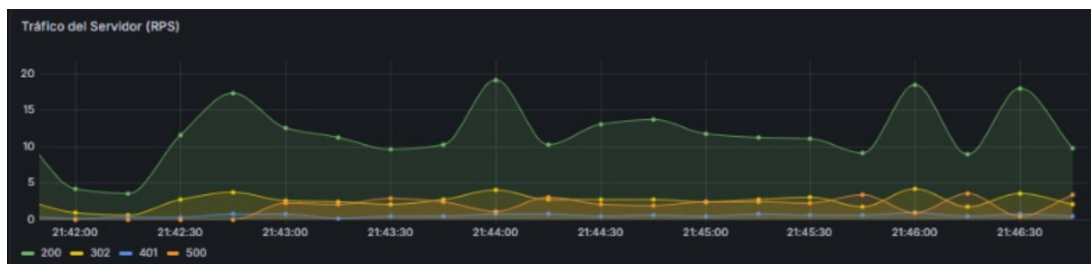


Figura 1: Gráfica del tráfico del servidor (RPS)

Tipo: Time Series

PromQL:

```
sum(irate(http_requests_total[1m])) by (status_code)
```

Muestra el número de peticiones por segundo (Requests Per Second) recibidas por el servidor, desglosadas por código de respuesta HTTP. Permite observar la carga total del sistema y distinguir rápidamente entre tráfico exitoso y erróneo.

4.2. Tráfico por Endpoint

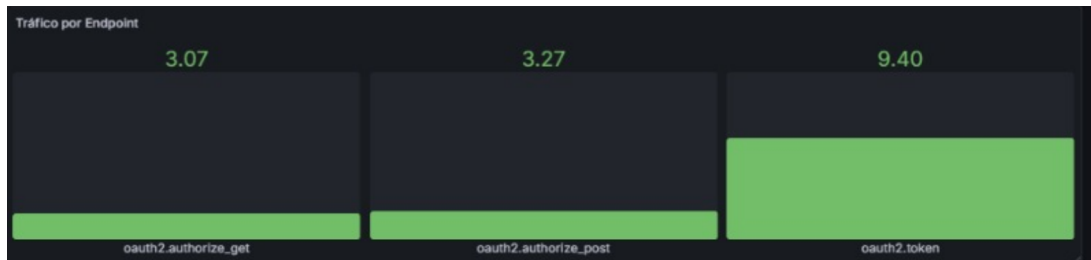


Figura 2: Gráfica del tráfico por cada Endpoint

Tipo: Stat Panels

PromQL:

```
sum(irate(http_requests_total[1m])) by (endpoint) > 0
```

Presenta el RPS por cada endpoint principal. En la prueba realizada destaca el alto volumen de peticiones al endpoint `/token`, seguido de `/authorize`, lo que concuerda con los pesos definidos en Locust.

4.3. Consumo de CPU del Backend

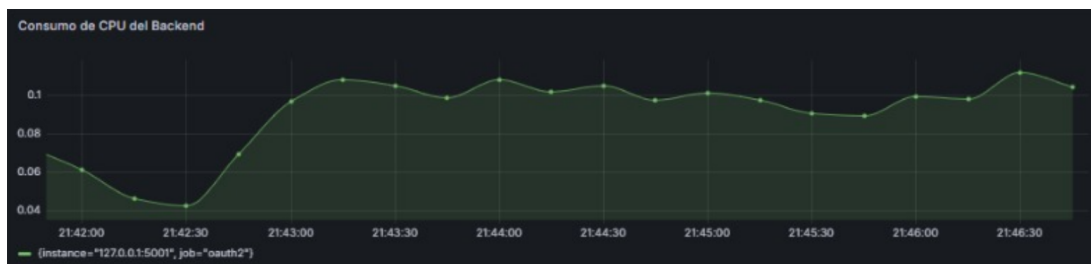


Figura 3: Gráfica del consumo de CPU del Backend

Tipo: Time Series

PromQL:

```
rate(process_cpu_seconds_total[1m])
```

Representa el consumo de CPU del proceso de la aplicación Flask. Permite correlacionar el aumento de tráfico con el uso de recursos del servidor y evaluar la eficiencia del backend bajo carga.

4.4. Latencia P95



Figura 4: Gráfica de la latencia en el percentil 95

Tipo: Time Series

PromQL:

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[1m])) by (le))
```

Muestra el percentil 95 de la latencia de todas las peticiones HTTP. Es un indicador clave de rendimiento, dado que no tiene en cuenta los outliers más altos (subirían mucho la media). Se estableció un umbral de advertencia visual en Grafana si el P95 superaba los 250ms (que en este caso es ampliamente superado), tal y como sugerían los requisitos para representar un tiempo de espera que el usuario puede percibir.

4.5. % de Errores del Servidor

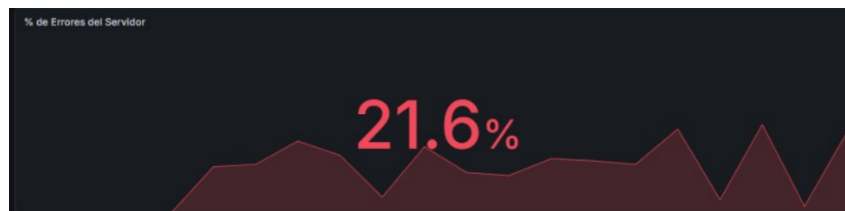


Figura 5: Gráfica del porcentaje de errores del servidor

Tipo: Stat + Time Series

PromQL:

```
sum(irate(http_requests_total{status_code=~"5.."}[1m]))  
/  
sum(irate(http_requests_total[1m])) * 100
```

Calcula el porcentaje de errores de servidor (códigos 5xx). Métrica importante para detectar cuándo el servidor empieza a someterse a una carga que directamente no puede ir despachando ni teniendo un tiempo de respuesta elevado.

4.6. Emisión de Tokens en tiempo real

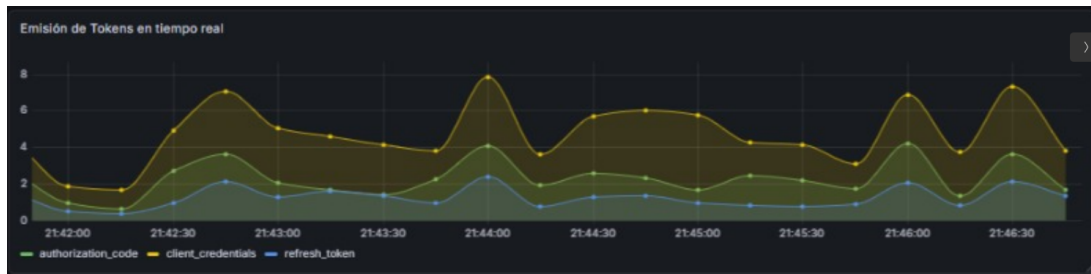


Figura 6: Gráfica de la emisión de Tokens en tiempo real

Tipo: Time Series

PromQL:

```
sum(irate(oauth2_tokens_issued_total[1m])) by (grant_type)
```

Visualiza la tasa de emisión de tokens por tipo de flujo (`client_credentials`, `authorization_code` y `refresh_token`). Permite verificar que los tres flujos OAuth2 se están ejecutando según los pesos configurados en las pruebas de carga.

4.7. Flujos de Autenticación Iniciados

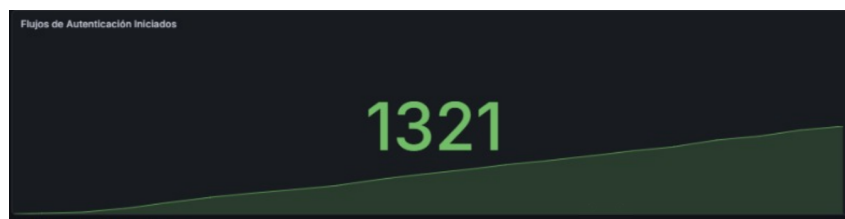


Figura 7: Gráfica de los flujos de autenticación iniciados

Tipo: Stat con tendencia

PromQL:

```
sum(oauth2_flows_total{status="started"})
```

Indica el número total acumulado de flujos de autenticación iniciados durante la prueba. En la captura se registraron 1321 flujos, reflejando una actividad intensa del sistema.

4.8. Tasa de Éxito de Autenticación

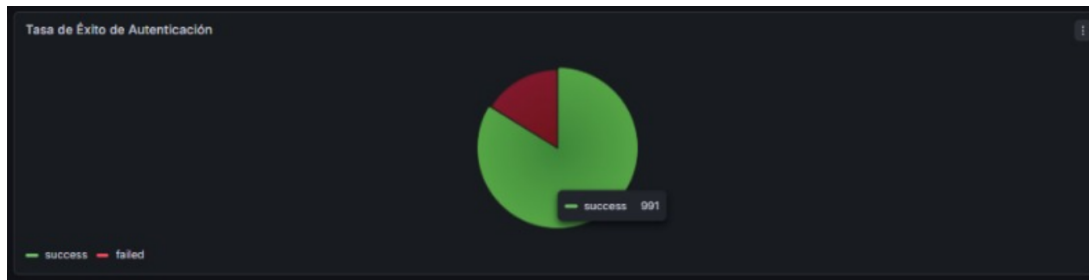


Figura 8: Gráfica del diagrama de sectores de la tasa de éxito de las autenticaciones

Tipo: Pie Chart

PromQL:

```
sum(oauth2_flows_total{status!="started"}) by (status)
```

Representa la proporción de flujos de autenticación exitosos frente a fallidos. El gráfico muestra una mayoritaria tasa de éxito (en verde), siendo los fallos mayoritariamente intencionados para simular comportamiento real de usuarios.

5. Locust para Realización de pruebas

5.1. Descarga e información relevante

Se descargó Locust y se creó un fichero `locustfile.py` inicial tal y como se indicaba en el enunciado de la práctica. Con esta configuración inicial ya completada, se desarrolló un script de inyección de tráfico basado en la librería Locust, en el cual se definieron tres tareas posibles para los usuarios (una por cada flujo de uso), siendo que en el 50 % de los casos el usuario virtual hace la acción de pedir credenciales (dado que hemos considerado que es la acción más pesada y relevante), y los otros dos flujos de ejecución un 33,33 % la comprobación del **authorization code** y un 16,66 % el flujo de **refresh token** (siendo la tarea que menos se realizará).

Antes de lanzar los programas de `app.py` y `locust`, hay que exportar las variables de entorno **SERVICE_ACCOUNT_SECRET** y **WEB_APP_SECRET** con las contraseñas obtenidas de la ejecución del fichero `seed.py` (cuya última salida (*claves vigentes*) ha sido guardada en el fichero **credencialesActuales.txt**).

Para garantizar el máximo rendimiento del cliente generador de carga, se heredó de la clase `FastHttpUser`. Los usuarios virtuales simulaban el ataque contra el endpoint `/token` inyectando credenciales reales de una *Service Account* obtenida de la base de datos.

5.2. Pruebas tipo identificadas

Las pruebas realizadas fueron especialmente tres:

- **Prueba nominal (15-usuarios | 5-ramp up):** Esta fue una prueba inicial de baja carga para comprobar el correcto funcionamiento y latencia del P95 aceptable, desde las tareas creadas en "locustfile.py", pasando por la recolección esperada de métricas, hasta la representación de dichas métricas en timeSeries en Grafana.
- **Prueba de carga (50-usuarios | 5-ramp up):** Esta prueba trata de ver cómo rinde el sistema bajo una carga de trabajo considerable, y parecida a la que se estima que será la carga de trabajo habitual del backend.
- **Prueba de estrés (75-usuarios | 5-ramp up):** Esta prueba trata de averiguar hasta qué punto de carga puede seguir funcionando el sistema. Averiguamos que un porcentaje elevado de peticiones empiezan a no poder resolverse y en 75 usuarios hemos encontrado una buena prueba de estrés, dado que puede seguir rindiendo, pero extremadamente exigido.
- **Prueba de estrés extrema (800-usuarios | 5-ramp up):** En este caso una prueba que pretendía ser de 80 usuarios accidentalmente se ejecutó con 800, cuyos efectos luego explicaremos más detenidamente. Como era de esperar tantos usuarios comprometieron la disponibilidad del servicio de backend (provocamos un ataque DDoS (Denegación de Servicio) autoprovocado y el servidor de las máquinas virtuales dejó de funcionar).

5.3. Descripción flujos de comportamiento

Los principales flujos de comportamiento que pueden seguir los usuarios virtuales (con las probabilidades ya explicadas) son los siguientes:

- **flow _authorization_code** (peso 2)
Simula el flujo completo *Authorization Code* tal como lo realizaría un usuario humano a través de un navegador:
 - Solicita la página de login (`/authorize GET`).
 - Envía credenciales (15 % de los intentos usan contraseña incorrecta para simular errores reales).
 - Extrae el `code` de la redirección.
 - Intercambia el código por un token de acceso (`/token`).
- **flow _client_credentials** (peso 3)
Simula el flujo *Client Credentials*, típico de comunicaciones máquina a máquina. Realiza directamente una petición a `/token` utilizando un *service-account* con sus credenciales. Es la tarea más frecuente, representando el caso de uso más común en entornos backend.

- **flow_refresh_token** (peso 1)

Simula la renovación de tokens. Solo se ejecuta si el usuario virtual ya dispone de un **refresh_token** (obtenido previamente en el flujo Authorization Code). Envía el refresh token a **/token** y actualiza el token en memoria en caso de éxito.

5.4. Configuración general

Los usuarios virtuales esperan entre 1 y 3 segundos entre tareas (**wait_time = between(1, 3)**). Al iniciar, cargan automáticamente los secretos de clientes desde el archivo **credencialesActuales**, generado por el script de seed. Esto permite ejecutar pruebas realistas y repetibles que cubren tanto flujos interactivos como automatizados.

Esta combinación de tareas permite generar una carga representativa del uso real del servidor OAuth2, combinando autenticaciones de usuarios y de servicios.

```
import os
import random
from urllib.parse import urlparse, parse_qs
from locust import FastHttpUser, task, between

...

class OAuth2User(FastHttpUser):
    wait_time = between(1, 3) # Simulacion de latencia

    ...

    @task(3)
    def flow_client_credentials(self):
        """Simula una app backend pidiendo acceso directamente."""
        payload = {
            "grant_type": "client_credentials",
            "client_id": "service-account",
            "client_secret": SERVICE_ACCOUNT_SECRET,
            "scope": "read",
        }
        with self.client.post(
            "/token", data=payload, name="/token (Client Credentials)", catch_response=
True
        ) as response:
            if response.status_code == 200 and "access_token" in response.text:
                response.success()
            else:
                response.failure(f"Fallo emitiendo token: HTTP {response.status_code}")
        ...
```

Definición del comportamiento de ataque (locustfile.py)

6. Resultados y Evaluación de Rendimiento

Las pruebas se ejecutaron mediante túneles SSH hacia una máquina virtual remota (`virtual.lab.inf.uva.es`), controlando la ejecución desde las interfaces web locales.

6.1. Prueba de Carga (50 Usuarios Concurrentes)

Con una simulación moderada de 50 usuarios concurrentes (tasa de aparición de 5 us/seg), el sistema demostró un comportamiento cercano a sus límites de arquitectura de desarrollo.

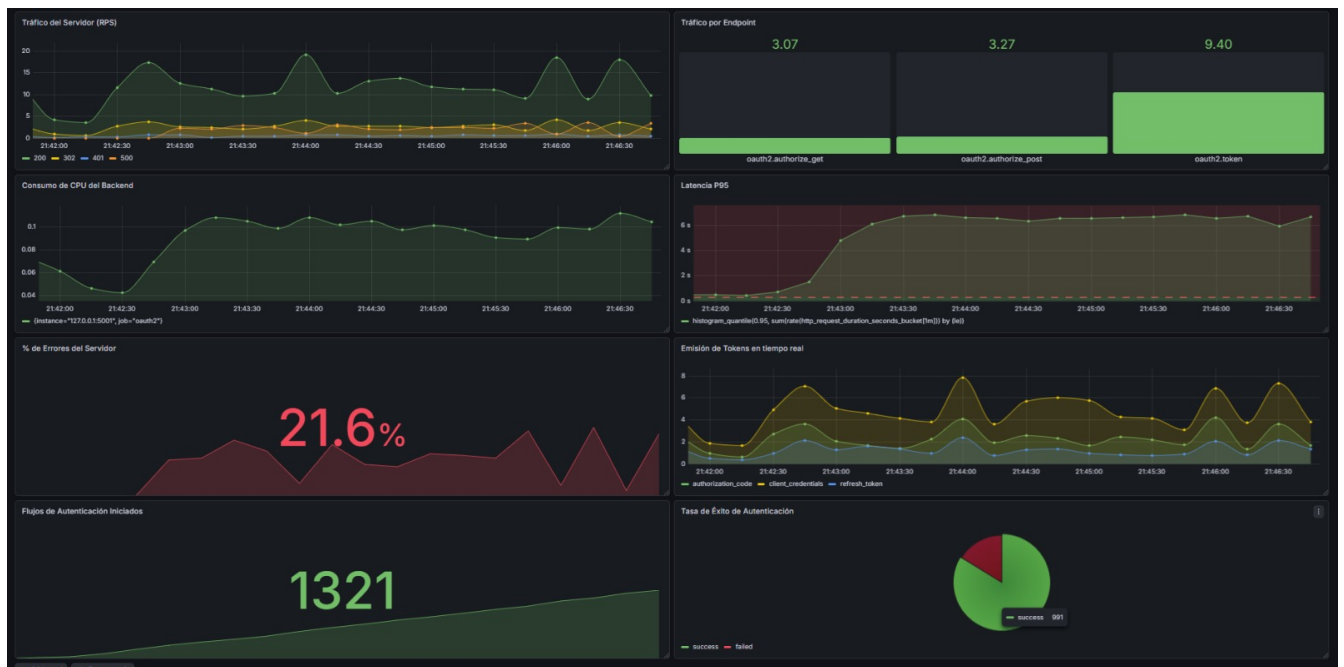


Figura 9: Gráfica de la emisión de Tokens en tiempo real

- **Tasa de Procesamiento (RPS):** El sistema logró mantener un flujo considerablemente estable de **15-20 RPS** (Peticiones Por Segundo).
- **Latencia (Percentil 95 %):** Locust registró un P95 de **4-6 segundos**, lo que supera muy ampliamente el objetivo teórico de 250ms.
- **Tasa de Errores:** El panel registró alrededor de un **21 %** de errores, un ratio intolerable para la mayoría de aplicaciones.

6.2. Prueba de Estrés y Punto de Ruptura (800 Usuarios)

Para encontrar el colapso del sistema, se reconfiguró Locust inyectando 800 usuarios concurrentes (*spawn rate* de 50). El resultado fue la saturación inmediata de los recursos computacionales.

Los síntomas documentados durante este ataque de denegación de servicio (DDoS) auto-provocado (antes del colapso total del sistema y la imposibilidad de seguir consultando los datos) fueron:

1. **Degradación extrema de Latencia:** El tiempo de respuesta (P95) para emitir un token se disparó hasta los **14.000 ms** (14 segundos).
2. **Fracaso de Conexiones:** Se alcanzó una tasa de error *Failures* del **60 %** en Locust, indicando que Flask rechazaba la mayoría de peticiones.
3. **El Apagón de Telemetría (No Data):** Todas las métricas de Grafana sufrieron cortes abruptos. Esto se debe a que el hilo principal de Python estaba tan asfixiado procesando solicitudes a `/token` que fue incapaz de responder a las llamadas de *scraping* de Prometheus a la ruta `/metrics`, provocando caídas por *Timeout*. El colapso del servidor llegó a tal extremo que bloqueó temporalmente el demonio SSH de la propia máquina virtual.

7. Conclusiones Finales

La inyección de telemetría directamente en el código fuente resulta indispensable para diagnosticar el comportamiento del software en tiempo real. Mediante la instrumentación con Prometheus y Grafana, no solo medimos el tráfico HTTP, sino que cuantificamos eventos de la lógica de negocio (tokens emitidos, validación de credenciales...).

La prueba de estrés ha demostrado empíricamente por qué la advertencia nativa de Flask (*"WARNING: This is a development server. Do not use it in a production deployment"*) debe respetarse estrictamente. El servidor WSGI *Werkzeug* y el motor de base de datos *SQLite* colapsan por inanición de recursos y bloqueos de escritura secuencial ante volúmenes de tráfico que un servidor de producción moderno debería asimilar sin dificultad.

Para habilitar este desarrollo en un entorno real, será mandatorio desplegar la aplicación tras un servidor de aplicaciones multihilo como **Gunicorn** y migrar la persistencia de datos a un gestor concurrente como **PostgreSQL**.